1. **To create and manage a GitHub repository and synchronize local and remote repositories using basic Git commands.**

Objective

To equip students with core competencies in remote version control management using Git and GitHub. This module establishes practical fluency in linking local repositories to remote cloud hosts, publishing code snapshots, cloning existing repositories, and synchronizing changes across distributed development environments.

Learning Outcomes

By the end of this lab, students will be able to:

- **Create and configure** a remote repository on GitHub using the web interface.
- **Link** a local Git repository to a remote tracking server using git remote add.
- **Publish** local commits to a remote branch using git push and set upstream tracking with -u.
- **Clone** a remote repository to create a local working copy using git clone.
- **Synchronize** local branches with remote updates using git fetch and git pull.
- **Inspect and manage** remote connections using git remote -v.

Prerequisites & Authentication Note

Before starting, ensure you have:

1. **Installed Git** on your local machine.
2. **A GitHub Account** (sign up at github.com).
3. **Authentication Setup**: GitHub requires either an **SSH Key** or a **Personal Access Token (PAT)** for authentication over HTTPS when executing git push.

Procedure

Creating a Remote Repository on GitHub: GitHub Setup.

Create an empty target repository on GitHub's web interface to host your project code.

1. Log in to **GitHub** and click the **+** icon in the upper-right corner → Select **New repository**.
2. Set **Repository name**: github-sync-demo.
3. Choose **Public** or **Private**.
4. **Uncheck** "Add a README file", "Add .gitignore", and "Choose a license" (keeping the remote repository completely empty allows clean initial syncing).
5. Click **Create repository**.
6. Copy the provided HTTPS or SSH repository URL (e.g., https://github.com/username/github-sync-demo.git).

Creating and Linking Local Repository: Local Initialization.

Initialize a local directory on **Linux**, commit sample files, and bind the local repository to your new GitHub remote.

Create and move into project workspace

mkdir ~/github-sync-demo

cd ~/github-sync-demo

Initialize local Git repository

git init

Create initial sample file

echo "# GitHub Synchronization Lab" > README.md

echo "console.log('Sync complete!');" > index.js

Stage and commit

git add .

git commit -m "feat: Initial project setup"

Ensure current branch is named 'main'

git branch -M main

Add the remote repository alias named 'origin'

git remote add origin https://github.com/YOUR_USERNAME/github-sync-demo.git

Verify remote connection details

git remote -v

Pushing Local Commits to GitHub: Publishing Code.

Upload your local commits to the remote repository and set the default upstream branch.

Push local main branch to remote 'origin' and set upstream tracking (-u)

git push -u origin main

ping -c 3 github.com

Note: If prompted for credentials when using HTTPS, enter your GitHub username and paste your **Personal Access Token (PAT)** as the password.

Cloning the Remote Repository: Replication.

Simulate working from a second machine by cloning the remote GitHub repository into a separate folder.

Navigate to home directory

cd ~

Clone the remote repository into a new folder 'cloned_project'

git clone https://github.com/YOUR_USERNAME/github-sync-demo.git cloned_project

Move into cloned project directory

cd ~/cloned_project

Verify commit history matches original project

git log --oneline

Making Changes in Cloned Copy and Pushing: Simulating Remote Changes.

Create new commits in the cloned workspace and push them to GitHub.

Make modifications inside cloned copy

echo "New line added from secondary environment." >> README.md

Stage, commit, and push changes

git add README.md

git commit -m "docs: Update README from secondary workspace"

git push

Pulling Remote Updates to Primary Workspace: Synchronization.

Return to your original local repository and retrieve the updates pushed from the second environment.

Return to primary project directory

cd ~/github-sync-demo

Fetch and merge remote changes in a single command

git pull origin main

Inspect README.md to verify new contents arrived

cat README.md

Verify local log updated smoothly

git log --oneline

command and Description

Command	**Action**	**Key Options / Notes**
git remote add <name> <url>	Binds a remote URL to a short local alias (usually origin)	Useful for linking local repos to GitHub/GitLab
git remote -v	Lists all configured remote aliases and their URLs	Displays fetch and push endpoints
git push <remote> <branch>	Uploads local branch commits to the remote repository	Use -u (--set-upstream) on first push

Command	Action	Key Options / Notes
git clone <url> [dir]	Downloads a complete remote repository and sets up tracking	Automatically adds origin remote alias
git fetch <remote>	Downloads updates from remote without merging into working files	Safe way to inspect incoming changes
git pull <remote> <branch>	Fetches remote changes **and** automatically merges them into current branch	Combination of git fetch + git merge